

01_model&init 大纲

1.Model OI介绍	1
2.调用模型	1
2.1 模型的分类方式	1
2.2 模型调用的角度	1
2.2.1 角度1：按功能列举	1
类型1：非对话模型（LLMs、Text Models）	1
类型2：对话模型 ChatModel	2
类型3：Embedding模型	3
1) OpenAI的嵌入模型	3
2) 百炼平台Qwen的嵌入模型	3
3) 硅基流动的Qwen Embedding模型,采用OpenAIEmbedding兼容接口	4
2.2.2 角度2：参数位置不同举例	4
方式1：API key和url硬编码	5
方式2：通过环境变量配置	5
方式3：配置文件方式	6
2.2.3 角度3：各个平台的API调用	7
2.3 大模型初始化方式	7
方式一、用ChatOpenAI调用，一般各厂商模型可以兼容	8
方式二、用对应的接口调用，ChatDeepseek、ChatOpenrouter	9
方式三、用init_chat_model() ---推荐	9
调用本地模型	13
1) 用ChatOllama	13
2) 使用init_chat_model()	13

02_modelinvocation 大纲

1.invoke()输入参数	1
1) 字符串类型	1
2) 字典列表（推荐）	2
关于多轮对话与上下文记忆	3
非同一个message同时传入，无记忆	3
传递记忆	3
3) 消息对象列表	4
传递记忆	5
2.invoke()返回值	6
美化输出	7
获取返回值的相关信息	8
3.流式输出stream()	9
流式输出与非流式输出	9
流式调用LLM获取响应	10
4.批量调用batch()	10
举例1 一次性接受所有响应.batch()	10
举例2：按照完成的顺序接受响应 .batch_as_completed()	11
举例3： batch和循环invoke性能对比	12
举例4： 消息对象列表batch	13
5.同步调用与异步调用	14
1) 同步调用	14
2) 异步调用	15
举例1： 异步ainvoke()	16
举例2： 异步astream()	17
举例3： 异步abatch()	19
3) 判断什么时候用什么调用方式	21
6.拓展知识	21
1) 输出美化	21
2) 模型的profile属性的使用	21
举例1： DeepSeek官网模型能力画像--无	21
举例2： SILICON平台模型能力画像--无	22
举例3： Opentouter平台模型能力画像	22
7.模型初始化参数（完整版）	24
1) 初始化参数	24

举例1: ChatDeepSeek()\ChatOpenAI()	24
举例2: init_chat_model()	25
2) dict_keys中的几类参数	25
(1) 客户端与连接参数 (Networking)	25
(2) 模型推理参数 (Model Inference)	26
(3) LangChain框架通用的参数	26
(4) 高级与特定扩展参数	27
model_kwargs	27
extra_body	27
(5) 需要记住哪些参数?	31
8.模型调用 (invoke、stream、batch等) 中的config参数	32

03_LangSmith&Langfuse 大纲

1.LangSmith的使用	1
----------------	---

2.Langfuse本地化部署及使用	5
1) Langfuse本地部署	5
2) 代码tracing举例	6

04_messages 大纲

1.认识消息(Messages)	1
1) 消息内部结构	1
2) 消息类型	1
3) 消息格式	1
格式1: JSON格式	1
格式2: 对象格式	2
HumanMessage的使用	3
ToolMessage的使用	3
4) 对话历史管理	5
5) 对话历史优化	6
6) 多轮对话聊天机器人	7
7) 拓展-消息属性: content、content_blocks	10
(1) content 的使用	10
(2) content_blocks的使用	10
输入格式化: anthropic兼容	11
输入格式化: openai兼容	12
输出格式化: deepseek	14

05_prompttemplates 大纲

1.提示词模板(Prompt Templates)	1
1.1 复习: str.format()	1
1) 带有位置参数的用法	2
2) 带有关键字参数的用法	2
3) 使用字典解包的方式	2
4) 字符串拼接方式	2
1.2 提示词模板	3
1) PromptTemplate --langchain1.0之后弱化了	3
(1) 两种实例化方式	4
方式1: 构造方法 format()	4
举例1: 单变量	4
举例2: 定义多变量	4
方式2: 调用from_template() 推荐!!!	4
(2) 两种新的结构形式	5
形式1: 部分提示词模版	5
方式1: 实例化过程中使用partial_variables变量	5
方式2: 使用 PromptTemplate.partial() 方法创建部分提示模板	5
形式2: 组合提示词(了解)	6
(3) 给变量赋值的两种方式: format() 与 invoke()	7
(4) 结合LLM调用	8
2) ChatPromptTemplate	8
(1) 两种实例化方式	9
方式1: 使用实例初始化方法	9
方式2: 调用from_messages()--推荐	10
(2) 调用提示词模板的几种方法	10
方法1: invoke() ---返回ChatPromptValue	10
方法2: format() ---返回str	11
方法3: format_messages() ---返回消息构成的list	11
方法4: format_prompt() ---返回ChatPromptValue	11
如何实现ChatPromptValue与list[messages]、字符串之间转换?	12
(3) 更丰富的实例化参数类型	13
举例1: 元组列表	13
举例2: 字符串列表	14
举例3: 字典列表	14
举例4: 消息对象列表	15
举例5: BaseChatPromptTemplate参数列表	16
举例6: BaseMessagePromptTemplate参数列表	16
综合案例	17
(4) 结合LLM	18
(5) ChatPromptTemplate的高级特性	19
1) 部分变量的预填充: partial()	19
2) 消息占位符	20

JSON形式	20
插入消息列表: MessagesPlaceholder	21
3) 存储对话历史内容	22
4) 可复用的模板库	23
举例1: template.py文件声明模板库	23
举例2: 分类创建模板	24
5) 模板组合	24
方法1: 字符串组合	24
方法2: 使用+运算符	25
1.3 少量样本示例的提示词模板	25
1) 使用说明	25
2) FewShotPromptTemplate使用	25
举例1 未提供示例的情况	25
举例2 使用FewShotPromptTemplate	26
3) FewshotChatMessagePromptTemplate的使用	27
4) Example selectors(示例选择器)	29
举例1: SemanticSimilarityExampleSelector	29
举例2: 结合FewShotPromptTemplate使用	31
5) PipelinePromptTemplate(了解)	32
6) 自定义提示词模版(了解)	34
7) 从文档中加载Prompt(了解)	35
(1) yaml格式提示词	35
(2) json格式提示词	35
(3) 调用py文件提示词模板	36

06_tools 大纲

1.Tools概述	1
2.工具调用的方式	1
调用方式1：直接调用	1
调用方式2：绑定到模型（主流）	2
3.工具调用的整体流程	3
4.从Messages流转看工具使用	3
5.工具的定义方式1：不使用@tool的方式	6
1) 举例：不是有装饰器	6
6.工具描述的各部分详解	8
1) 了解convert_to_openai_tool	8
2) description说明	9
3) 参数说明	9
4) 参数类型说明	10
5) 参数默认值说明	11
举例1：有默认值	11
举例2：无默认值	11
7.工具的定义方式2：使用@tool装饰器(推荐)	12
自定义工具描述：description	12
情况1：仅提供docstring信息	12
情况2：添加工具描述：description	13
情况3：解析docstring：parse_docstring	13
更改工具名称：name_or_callable	16
自定义args_schema	17
方式1：使用Pydantic模型定义	17
pydantic类型的定义	17
① BaseModel基类	17
② Field	18
③ Literal	18
使用Pydantic定义args_schema	19
方式2：使用Json Schema定义	20
8.工具的应用案例	22
案例1：使用args_schema	22

案例2：撰写docstring	24
案例3：多工具调用	26
案例4：多工具调用	30
9.拓展：强制使用工具	31
1) tool_choice参数说明	31
2) none值举例	32
3) auto值举例	32
举例1：需要调用工具的场景	32
举例2：不需要调用工具的场景	33
4) required值举例	33
举例1：需要调用工具的场景	33
举例2：不需要调用工具的场景	33
5) 强制调用特定的工具	35
10.实践经验总结	36
1) 清晰的描述	36
2) 功能单一，每个工具做一件事	36
3) 如何处理工具失败？	36
4) 返回字符串	37
5) 选择同步 vs 异步	38

07_output 大纲

1.结构化输出	1
2.传统方式 vs 结构化输出	1
1、传统的几种方式（繁琐、不推荐）	1
2、结构化输出（简洁）	2
3、结构化输出模式	3
3.四种模式的使用	3
1) 模式1：Pydantic	3
1、基本使用	3
2、高级特性	5
情况1：可选字段	5
情况2：默认值	6
情况3：枚举类型	8
方式1：Enum	8
方式2：Literal	9
情况4：列表提取	10
情况5：嵌套结构	11
情况6：限制条件	14
3、工作流程图解	15
第1步：定义结构	15
第2步：协议转换	15
第3步：模型交互与强约束	16
第4步：自动解析与验证	16
模式2：TypedDict	16
1) 什么是 TypedDict	16
1、普通 dict 没有类型信息：	16
2、TypedDict 可以进一步说明：	16
2) 基本使用	17
举例1：返回简单结构	17
举例2：嵌套结构的使用	18
模式3：JSON Schema	20
举例1：返回简单结构	20
举例2：返回嵌套结构	21
模式4：@dataclass	22
4.关于类型校验	24
1) fake server	24
2) 四种模式的校验	26
1.Pydantic	26
2.TypedDict	26
3.JSON schema	27

5.获取结构化结果方式	28
1) 使用with_structured_output	29
2) 使用输出解析器(不推荐)	30

08_agent 大纲

1.智能体	1
1.什么是agent	1
2.Agent的核心组件	1
3.Agent创建与调用	1
1) 历史上的调用	1
2) 全新的调用	2
2.Agent的基本用法1：模型的传入方式	3
2.1 传入模型字符串	4
2.2 传入模型对象	4
3.Agent的基本用法2：如何调用Agent	5
4.Agent的基本用法3：绑定工具	10
1) 基本用法	11
举例1：绑定一个工具	11
举例2：接入Langchain内置工具	15
举例3：绑定多个工具	21
2) 工具调用流程分析	25
举例：用户问题：“找出当前最流行的无线耳机并检查库存”的任务。	26
3) 重试机制	27
4) 常见问题	29
问题1：Agent 如何选择工具？	30
问题2：Agent 为什么没有调用工具？	30
问题3：Agent 选错工具？	30
问题4：如何知道 Agent 何时完成？	31
问题5：Agent 可以调用多少次工具？	31
问题6：如何限制工具调用次数？	31
5.Agent的高级用法1：设置Agent名称	32
5.1 用法	32
5.2 经典使用场景	32
1. 流式输出归因	33
2. 消息身份标记	33
3. 调试与trace可读性	33
4. 组件化封装	33
5. 前端展示与运行态可观测性	33
6. 作为稳定的运行时身份标识	33
6.Agent的高级用法2：系统提示词	34

7.Agent的高级用法3：结构化输出	41
7.1 模型 vs Agent的结构化输出对比	41
7.2 结构化输出的4种策略	41
① ProviderStrategy	42
② ToolStrategy--推荐	43
③ type / AutoStrategy	44
④ None	45
7.3 ToolStrategy使用详解	46
1) 结构化输出：schema参数	47
输出模式1：Pydantic类型	48
输出模式2：TypedDict类型	51
输出模式3：JsonSchema类型	57
输出模式4：@dataclass类型	61
输出模式5：多schema联合模式	63
2) 自定义工具消息：tool_message_content参数	65
3) 错误处理：handle_errors参数	69
情况1：设置为True/False/固定字符串	70
1) handle_errors设置为True	73
2) handle_errors设置为False	74
3) handle_errors设置为固定字符串---“请检查输入数据”	74
情况2：设置为指定异常类型	81
情况3：设置为自定义错误处理函数	84
8.Agent的高级用法4：流式输出及模式	87
8.1 流式输出的说明	87
8.2 具体的输出模式	88
1) values输出模式	88
2) updates输出模式	97
3) messages输出模式	102
4) tasks输出模式	103
5) debug输出模式	117
6) checkpoints输出模式	131
7) custom输出模式	135
8.3 流式输出模式总结	137
9.实战：多功能智能助手	143
9.1 模型的初始化	143
9.2 工具的定义	143
9.3 agent的创建	147
9.4 主程序	148

09_middleware 大纲

1.中间件概述	1
1.1 什么是中间件	1
情况1：没有中间件的Agent架构：	1
情况2：添加中间件之后的Agent架构变为：	2
1.2 为什么需要中间件	3
1.3 中间件的分类	4
1.4 和模型供应商无关的内置中间件分类	5
类型1：成本与资源控制类	5
类型2：稳定性与容错保障类	6
类型3：安全与合规风控类	6
类型4：决策增强与智能编排类	6
类型5：执行能力扩展类	7
类型6：开发调试与测试辅助类	7
2、常用内置中间件的使用	7
2.1 SummarizationMiddleware中间件	7
2.1.1 参数说明	8
参数1：model —用于摘要的模型	8
参数2：trigger —摘要触发条件	8
参数3：keep —摘要时保留的原始消息	8
参数4：token_counter —统计token数量的函数	8
参数5：summary_prompt —摘要时的自定义提示词	8
参数6：trim_token_to_summarize —摘要时历史消息的最大token数	8
2.1.2 举例1 测试trigger、keep参数	9
2.1.3 举例2 测试summary_prompt参数	10
2.2 HumanInTheLoopMiddleware中间件	12
2.2.1 参数说明	12
参数1：interrupt_on —工具名和中断策略的映射	12
参数2：description_prefix —自定义中断描述	13
2.2.2 举例过程1：调用前中断	13
2.2.3 举例过程2：指明工具调用请求决策	19
2.3 PIIMiddleware中间件	22
2.3.1 参数说明	22
参数1：pii_type —检测的PII数据类型	22
参数2：strategy —处理PII信息的策略	22
参数3：detector —自定义PII检测函数或者正则表达式	22
参数4：apply_to_input —是否在调用模型前检测	23
参数5：apply_to_output —是否在模型调用后检测	23
参数6：apply_to_tool_results —是否在工具调用后检测其输出	23
2.3.2 举例1：使用内置检测器	23
2.3.3 举例2：自定义检测器/函数	24
2.4 TodoListMiddleware中间件	26

2.4.1 参数说明	26
2.4.2 案例设计	27
2.4.3 代码	27
1、环境准备	27
2、业务代码	28
3、调用及输出	30
4、分析	40
3、其它内置中间件	41
3.1 ModelCallLimitMiddleware中间件	41
举例1：整个会话限制-优雅退出	41
举例2：整个会话限制-抛异常	44
举例3：单次调用限制-优雅退出	48
举例4：单次调用限制-抛异常	53
3.2 ToolCallLimitMiddleware中间件	57
举例1：整个会话限制-优雅结束	57
举例2：整个会话限制-抛异常	60
案例3：单次调用限制-继续运行	64
3.3 ModelFallbackMiddleware中间件	69
举例1：实例设置models	69
举例2：create_agent中设置model	69
3.4 LLMTToolSelectorMiddleware中间件	70
举例1:max_tool=0,always_include=["get_weather"]	71
举例2:max_tool=0,always_include=["get_news"]	73
举例3:max_tool=0,always_include=["get_weather","get_news"]	75
举例4:max_tool=1,always_include=["get_weather"]	77
3.5 ToolRetryMiddleware中间件	79
举例1：带抖动	79
举例2：无抖动	81
3.6 ModelRetryMiddleware中间件	85
举例1：继续运行	85
举例2：抛异常	85
3.7 LLMTToolEmulator中间件	86
3.8 ContextEditingMiddleware中间件	89
1. 实验组-启用上下文编辑	89
2. 对照组-不裁剪上下文	90
3.9 FilesystemFileSearchMiddleware中间件	91
3.10 Shell tool中间件	94
3.11 Filesystem中间件	94
3.12 Subagent中间件	94
4、多个中间件组合及执行顺序	94
5、自定义中间件	96

5.1 什么是hook函数(钩子函数)	96
5.2 LangChain的hook函数分类	98
类型1: Node-style hooks(节点风格钩子)	98
类型2: Wrap-style hooks(包装风格钩子)	98
5.3 Node-style hooks函数用法	98
5.3.1 基本用法	98
1. 基于装饰器实现	98
2. 基于类实现	100
5.3.2 两种方法的统一	102
5.3.3 参数说明	103
5.3.4 返回值说明	104
5.3.5 装饰器参数: can_jump_to	104
1. 基于装饰器实现	104
2. 基于类实现	120
5.4 Wrap-style hooks函数用法	114
5.4.1 基本用法	114
1. wrap_model_call	126
① 基于装饰器实现	98
② 基于类实现	100
场景1: 重试逻辑	85
场景2: 响应缓存	117
场景3: 修改系统提示	118
2. wrap_tool_call	118
① 基于装饰器实现	98
② 基于类实现	100
5.4.2 两种方法的统一	122
5.4.3 参数说明	122
5.5 装饰器和类的选择	122
情况1: 中间件只用一个钩子函数, 推荐用装饰器, 需要多个钩子函数推荐类写法	122
1. 使用装饰器实现	123
2. 使用类实现	123
情况2: 复杂配置推荐用类实现	124
情况3: 跨项目复用推荐用类写法	125
5.6 hook函数执行顺序(重要)	126
1. 代码	126
2. 分析	128

10_context&memory 大纲

1、概述	1
1.1 为什么需要记忆(Memory)	1
情况1：不同messages	1
情况2：同一个messages列表	2
1.2 如何解决记忆问题	3
1.2.1 上下文工程	3
1.2.2 上下文类型及相关的API	3
1.3 LangChain的记忆	3
1.3.1 记忆的分类	4
1.3.2 记忆的管理	4
2、短期记忆	4
2.1 基于内存的持久化器	5
2.1.1 举例1：没有记忆	5
2.1.2 举例2：拥有记忆	5
步骤1：第一二轮对话	5
步骤2：第三轮对话	8
步骤3：更新线程ID	12
2.1.3 关键步骤说明	13
场景1：多用户聊天	13
场景2：同一用户的不同任务	14
2.1.4 工作原理	14
2.1.5 常见问题	15
1、为什么 Agent 不记得？	15
2、InMemorySaver 会丢失数据吗？	15
3、内存会无限增长吗？	15
4、如何清空某个会话的历史？	16
2.2 基于外部存储介质的持久化器	16
2.2.1 数据库环境准备	16
2.2.2 代码实现	16
2.2.3 查看持久化数据	20
需求1：查看所有数据库	20
需求2：查看所有schema	20
需求3：查看当前所处的schema	21
需求4：查看当前schema下的所有表	21
2.3 对比两种方式	21
举例1：基于内存存储	21
举例2：基于外部存储器存储	23
2.4 记忆治理策略（上下文管理）	27
2.4.1 消息裁剪	28
2.4.2 消息删除	31
2.4.3 摘要	35

2.4.4 自定义过滤策略	39
2.5 了解：state的理解	44
3、长期记忆	47
3.1 基本理解	47
3.1.1 什么是长期记忆	47
3.1.2 类型划分	48
3.1.3 存储架构	49
3.2 基础API的使用	50
3.2.1 put()/get()：写入/读取API	50
① put()源码剖析	50
② get()源码剖析	51
③ 举例1：基于InMemoryStore	51
举例1：添加	51
举例2：更新	52
④ 举例2：基于PostgresStore	52
举例1：添加	52
举例2：更新	52
3.2.3 search()：检索API	53
① 源码剖析	53
② 举例1：按照namespace前缀搜索	53
③ 举例2：按照filter过滤	55
④ 举例3：按照语义搜索	56
举例1：自定义嵌入函数	56
举例2：使用嵌入模型	58
3.3 在Agent运行图中访问长期记忆	60
3.3.1 在工具中访问长期记忆	60
① 基于InMemoryStore	60
② 基于PostgresStore	62
3.3.2 在中间件中访问长期记忆	65
① Node-style hooks中访问	65
② Wrap-style hooks中访问	65
1. wrap_model_call	65
2. wrap_tool_call	66
3.4 何时写入记忆	67
1. 在主流程里写（hot path）	67
2. 在后台写（background）	67
4、课后阅读：Static runtime Context	67
4.1 中间件中访问	68
4.1.1 Node-style hooks	68
4.1.2 Wrap-style hooks	71
① wrap_model_call	71
② wrap_tool_call	76
4.1.3 特殊的便捷中间件dynamic_prompt	79
4.2 工具中访问	83

11_RAG(Retrieval) 大纲

1、Retrieval模块的设计意义	1
1.1 大模型的局限	1
1) 知识滞后	1
2) 知识缺失	1
3) 幻觉	1
1.2 什么是RAG	1
1.3 RAG优缺点	4
RAG的优点	4
RAG的缺点	4
1.4 RAG工作流程	4
环节1: Source (数据源)	4
环节2: Load (加载)	5
环节3: Transform (转换)	5
环节3.1: Text Splitting (文档拆分)	5
环节4: Embed (嵌入)	6
环节5: Store (存储)	7
环节6: Retrieve (检索)	7
2、详细使用流程	7
2.1 环境准备	7
2.1.1 安装依赖	7
2.1.2 准备数据	8
2.2 文档加载器 Document Loaders	8
2.2.1 加载txt	8
2.2.2 加载CSV	9
2.2.3 加载JSON	10
举例1: 使用JSONLoader文档加载器加载	10
举例2: 提取04-response.json文件中指定的文本	11
加载json文件中所有的数据	11
加载json文件中employees[]中的name字段	11
提取04-response.json文件中嵌套在data.items[].content的文本	12
提取04-response.json文件中嵌套在data.items[]里的title、content和其文本	13
2.2.4 加载pdf	13
方式1: PyPDFLoader	14
方式2: MinerU	16
2.2.5 加载word	18
2.2.6 加载Markdown	19
举例1: 使用UnstructuredMarkdownLoader加载md文	19
举例2: 精细分割文档, 保留结构信息	20
2.2.7 加载HTML(了解)	21
2.2.8 加载File Directory(了解)	24
2.2.9 了解: BaseLoader、Document类	25
BaseLoader类分析	25

Document类分析	26
2.3 文档切分器 Text Splitters	26
2.3.1 为什么分割/切分/分块？	26
2.3.2 Chunking拆分的策略	27
方法1：根据句子切分	27
方法2：按照固定字符数来切分	27
方法3：按固定字符数来切分，结合重叠窗口（overlapping windows）	27
方法4：递归字符切分方法	27
方法5：根据语义内容切分	27
2.3.3 几个常用的文档切分器的方法的调用	28
2.3.4 具体实现	28
① CharacterTextSplitter：Split by character	28
举例1：字符串文本的分割	28
举例2：指定分割符	29
举例3：指定分割符	30
② RecursiveCharacterTextSplitter：最常用	30
举例1：使用split_text()方法	31
举例2：使用create_documents()方法1	32
举例3：使用create_documents()方法2	34
举例4：使用split_documents()方法	41
举例5：自定义分隔符	43
③ TokenTextSplitter/CharacterTextSplitter：Split by tokens	44
举例1：使用TokenTextSplitter	45
举例2：使用CharacterTextSplitter	46
④ SemanticChunker：语义分块	47
1. breakpoint_threshold_type（断点阈值类型）	48
2. breakpoint_threshold_amount（断点阈值量）	49
3. sentence_split_regex（句子切分的正则表达式）	49
⑤ HTMLHeaderTextSplitter(了解)	49
⑥ CodeTextSplitter(了解)	50
举例1：支持的语言	51
举例2：python语言	51
⑦ MarkdownTextSplitter(了解)	51
举例：md数据类型	51
2.4 文档嵌入模型 Text Embedding Models	52
2.4.1 嵌入模型概述	52
2.4.2 嵌入模型选型与初始化	54
选型：使用硅基流动平台的嵌入模型	54
初始化方式	54
初始化方式1：init_embeddings()	54
初始化方式2：OpenAIEmbeddings()	54
2.4.3 句子的向量化（embed_query）	54
2.4.4 文档的向量化（embed_documents）	55
2.5 向量存储(Vector Stores)	56
2.5.1 向量数据库的理解	56
2.5.2 常用的向量数据库	57
2.5.3 案例：民法典客服知识库	57
① 全局配置	58
② 初始化Milvus	58
③ 初始化 Embedding 模型	59

④ 读取文档并切分	60
⑤ 生成向量并写入 Milvus	62
⑥ 初始化模型与Agent	64
⑦ 检索逻辑 (Retrieval)	64
⑧ 生产与回答生成	65

12_Linux服务器部署postgresql 大纲

1、腾讯云服务器开通	1
2、如何安装PostgreSQL	4
2.1 在Ubuntu系统安装PostgreSQL	5
1) 安装命令	5
2) 验证	5
3) 查看状态与端口占用	5
查看状态	5
查看端口占用	5
2.2 创建用户和数据库	5
1) 启动客户端	5
2) 创建用户	5
3) 创建数据库	6
4) 赋权	6
5) 查看数据表	6
2.3 测试URL	6
2.4 远程服务器链接额外操作	6
2.4.1 安全组放通5432端口	7
2.4.2 PostgreSQL监听所有IP	7
1) 查看相关配置文件路径	7
2) 更改配置文件	8
3) 重启服务	8
2.4.3 添加允许规则	8
1) 查看配置文件路径	8
2) 更改配置文件	8
3) 重启服务生效	9
4) 测试连接	9

13_Milvus使用指南 大纲

1、安装Docker DeskTop	1
1.1 下载安装包	1
1.2 设置Proxy	1
1.3 设置国内镜像（备选）	1
2、安装Milvus	2
2.1 standalone.bat 方式（不推荐）	2
2.2 docker compose方式	3
3、测试前准备工作	4
3.1 确保Milvus已启动	4
3.2 安装Milvus Python SDK	4
3.3 Milvus数据模型	5
3.3.1 整体结构	5
3.3.2 具体说明	5
4、基本用法	5
4.1 DDL操作	5
4.1.1 Database	5
1) 查看数据库	5
举例1：操作客户端	5
举例2：列出所有数据库	6
2) 创建新数据库	6
3) 删除数据库	6
4.1.2 Collection相关操作	6
1) 切换数据库	6
2) 查看数据库下的Collections	6
3) 创建collection	6
4) 删除collection	7
4.2 DML操作	7
4.2.1 嵌入模型初始化	7
4.2.2 准备collection	7
1) 创建collection	7
2) 查看collection元数据	7
4.2.3 准备数据	8
1) 准备原始数据	8
2) 生成嵌入向量	8
3) 查看生成的嵌入向量	9
4) 封装为可以插入的数据格式	9
4.2.4 写入数据	9
1) 写入数据	9
2) 手动flush	9

3、DQL操作	10
3.1 扫描数据	10
3.2 通过主键查询数据	10
3.3 相似度检索	11
1) 准备查询嵌入	11
2) 检索	11